

Memoria Técnica

cv-lit

Sistema de Monitorización Automática
de Línea de Costa

Guardamar del Segura · Alicante · España

Tech4D Lab
Universidad de Alicante
nickhernd

Junio 2026

Resumen

cv-lit es un sistema software de monitorización automática de línea de costa mediante visión por computador, desarrollado para el Ayuntamiento de Guardamar del Segura en colaboración con el Instituto de Ecología Litoral (IEL) y la Universidad de Alicante.

El sistema procesa imágenes oblicuas de **6 cámaras fijas Obscape** instaladas a lo largo del frente litoral (tramo norte-sur $\approx 3,5$ km), extrayendo automáticamente la línea de costa y proyectándola a coordenadas geográficas UTM EPSG:25830 (ETRS89 / UTM zona 30N).

Esta memoria recoge la arquitectura completa del sistema, los algoritmos de procesamiento de imagen (SIFT, RANSAC, SAM), el módulo de alineación masiva no-destructiva diseñado en la iteración de junio de 2026, la interfaz web de validación por lotes, los resultados de calibración geométrica obtenidos para las seis cámaras, y la documentación operacional completa para el despliegue y mantenimiento del sistema.

Índice general

1. Descripción del Proyecto	6
1.1. Contexto y motivación	6
1.2. Partes implicadas	6
1.3. Objetivos del sistema	6
1.4. Restricciones no negociables	7
1.5. Entregables	7
2. Infraestructura de Cámaras	8
2.1. Sistema Obscape	8
2.2. Distribución geográfica	8
2.3. Descarga automática de imágenes	8
2.3.1. Automatización con Cron	9
3. Arquitectura del Sistema	10
3.1. Stack tecnológico	10
3.2. Estructura de directorios	10
3.3. Pipeline de procesamiento	11
4. Calibración Geométrica	13
4.1. Fundamento matemático	13
4.1.1. Corrección de distorsión radial	13
4.1.2. Estimación RANSAC	13
4.2. Proceso de calibración por cámara	14
4.3. Resultados de calibración (junio 2026)	14
4.4. Imágenes de diagnóstico	15
4.5. Recalibración rápida	15
5. Módulo de Alineación Masiva	17
5.1. Motivación	17
5.2. Arquitectura de estados	17
5.2.1. Estructura del directorio temporal por job	18
5.3. Pipeline de alineación: algoritmo	18
5.3.1. Parámetros configurables	18
5.3.2. Estrategia de dos pasadas	18
5.3.3. Flujo completo del pipeline	19
5.4. Renderizado del mapa de delta	19
5.4.1. Capa 1 — Heatmap de diferencia absoluta	19
5.4.2. Capa 2 — Divergencia de bordes (overlay cian)	19
5.4.3. Capa 3 — Métrica escalar δ_{px} (flujo óptico Farneback)	20

5.5. API REST del módulo	20
5.6. Modelos de dominio	20
5.7. Flujo de commit (two-phase write)	21
6. Máscaras de Zonas Estables SIFT	22
6.1. Problema: objetos dinámicos en SIFT	22
6.2. Solución: máscaras de regiones estables	22
6.3. Zonas por cámara	23
6.4. Corrección CAM_1: problema de gaviotas	23
6.4.1. Coordenadas en píxeles (imagen 4608×2682)	24
6.5. Cómo ajustar una máscara	24
7. Interfaz Web	26
7.1. Arquitectura frontend	26
7.2. Componente BatchAlignment.vue	26
7.2.1. Variables de estado	26
7.2.2. Props y eventos	27
7.2.3. Layout de la fase reviewing	27
7.2.4. Indicadores del filmstrip	28
7.2.5. Polling de progreso	28
7.3. Flujo completo de uso	28
8. Región de Interés (ROI)	29
8.1. Criterios de delimitación	29
8.2. Coordenadas por cámara	29
9. Segmentación y Extracción de Línea de Costa	30
9.1. Segmentación semántica con SAM	30
9.2. Extracción de línea de costa	30
9.3. Proyección a EPSG:25830	31
9.4. Exportación GeoJSON	31
10. Validación y Métricas de Calidad	32
10.1. Estrategia de validación	32
10.2. Métricas	32
10.3. Criterios de aceptación	32
10.4. Factores de error a monitorizar	32
10.5. Validación en QGIS	33
11. Estado del Proyecto y Roadmap	34
11.1. Estado actual (junio 2026)	34
11.2. Completado en la iteración de junio 2026	34
11.3. Fase 3 — Automatización y robustez (mes 5)	35
11.4. Fase 4 — Entrega e integración (mes 6)	35
12. Guía de Operación	36
12.1. Inicio rápido	36
12.2. Descarga de imágenes	36
12.3. Calibración	36

12.4. Alineación masiva (CLI)	37
12.5. Pipeline de extracción	37
13.Instalación y Configuración	38
13.1. Requisitos del sistema	38
13.2. Instalación del entorno Python	38
13.3. Instalación del frontend	39
13.4. Configuración de la API Obscape	39
A. Convenciones de código y Git	40
A.1. Estrategia de ramas	40
A.2. Nomenclatura de ficheros	40
A.3. Proyección obligatoria	40
B. Glosario	41
C. Historial de cambios	42

Índice de figuras

3.1. Pipeline completo de procesamiento cv-lit	12
4.1. Diagnóstico de calibración — reproyección de GCPs (vectores $\times 10$) . .	15
5.1. Máquina de estados del módulo BatchAlignment	17
5.2. Flujo de commit no-destructivo	21
6.1. Comparativa de máscaras SIFT para CAM 1 (imagen 4608×2682 px) .	23

Índice de cuadros

1.1. Actores del proyecto	6
1.2. Atributos del GeoJSON de salida	7
2.1. Estaciones Obscape en Guardamar del Segura	8
3.1. Dependencias principales	10
4.1. Resultados de precisión por cámara	14
5.1. Parámetros del pipeline de alineación (<code>batch_alignment.py</code>)	18
5.2. Leyenda del heatmap HOT	19
5.3. Endpoints del router <code>/api/batch/</code>	20
6.1. Configuración de máscaras SIFT por cámara	23
6.2. Conversión fracción $\rightarrow$ píxeles para CAM_1	24
7.1. Props del componente BatchAlignment	27
7.2. Código de color en el filmstrip	28
8.1. ROIs preliminares por cámara (a ajustar con imágenes de producción) .	29
10.1. Umbrales de calidad del sistema	32
11.1. Estado de hitos del proyecto	34
13.1. Requisitos hardware y software	38
13.2. Parámetros de conexión Obscape	39
C.1. Historial de versiones del sistema	42

Capítulo 1

Descripción del Proyecto

1.1 Contexto y motivación

La playa de Guardamar del Segura es un espacio de alto valor ecológico y turístico sujeto a procesos de erosión y acreción que requieren seguimiento sistemático. La medición manual de la línea de costa mediante levantamientos GNSS es costosa, puntual y no escala para cubrir todo el frente litoral con frecuencia horaria.

El proyecto **cv-lit** surge para automatizar esta medición aprovechando la infraestructura de cámaras fijas ya desplegada por el servicio de vigilancia de la playa, convirtiendo cada fotografía en un dato georreferenciado de la posición de la interfaz arena-mar.

1.2 Partes implicadas

Cuadro 1.1: Actores del proyecto

Rol	Entidad
Cliente	Ayuntamiento de Guardamar del Segura
Datos GNSS (GCPs)	Instituto de Ecología Litoral (IEL)
Desarrollo	1 ingeniero a tiempo parcial (Tech4D Lab, UA)
Supervisión	Universidad de Alicante
Duración	6 meses (enero – junio 2026)

1.3 Objetivos del sistema

El producto final es un **GeoJSON proyectado en EPSG:25830** con los siguientes atributos obligatorios por cada detección:

Cuadro 1.2: Atributos del GeoJSON de salida

Campo	Tipo	Descripción
ID_Camara	Integer (1–6)	Identificador de cámara origen
Timestamp	ISO 8601 UTC	Fecha y hora de captura
Confianza_IA	Float (0–1)	Score de confianza del modelo SAM
Area_Seca_m2	Float	Superficie de playa seca (m ²)

1.4 Restricciones no negociables

- Toda implementación **debe** producir GeoJSON en **EPSG:25830**.
- Fase horaria prioritaria: **12:00h solar local**.
- Modelo de segmentación: **SAM** (Segment Anything Model, Meta AI).
- Cada cámara tiene su propio perfil de calibración *independiente*.
- Criterio de aceptación de precisión: **RMSE < 1,5 m** en condiciones estándar.

1.5 Entregables

1. Aplicación ejecutable configurada y documentada (esta memoria).
2. Perfiles de calibración por cámara (6 ficheros `.npy` / `.json`).
3. Interfaz web de operación y validación por lotes.
4. Informe de validación con métricas RMSE y MAE.

Capítulo 2

Infraestructura de Cámaras

2.1 Sistema Obscape

Las seis cámaras están gestionadas por la plataforma Obscape y se acceden via API REST autenticada. La tabla 2.1 lista los parámetros de registro de cada estación.

Cuadro 2.1: Estaciones Obscape en Guardamar del Segura

Cámara	Station	Referencia	Latitud (°N)	Longitud (°E)	Última act.
CAM 1	8213	PTM61471	38.110327	−0.643027	20-05-2026
CAM 2	8214	PTM61474	38.096291	−0.645725	20-05-2026
CAM 3	8212	PTM61473	38.087348	−0.646994	20-05-2026
CAM 4	8211	PTM61475	38.087385	−0.647078	20-05-2026
CAM 5	8209	PTM61472	38.076774	−0.648117	20-05-2026
CAM 6	8210	PTM61470	38.076796	−0.648114	20-05-2026

Nota geográfica

CAM 3 y CAM 4 son casi coincidentes ($\Delta \approx 8$ m) y cubren el mismo tramo desde ángulos ligeramente distintos. Lo mismo ocurre con CAM 5 y CAM 6. Esto permite comparación cruzada de la línea detectada para validación interna.

2.2 Distribución geográfica

Las cámaras se distribuyen de norte a sur a lo largo de $\approx 3,5$ km de frente litoral. La cobertura solapada garantiza continuidad en la extracción de línea de costa sin lagunas.

2.3 Descarga automática de imágenes

El módulo `acces_api/scheduled_download.py` implementa una **ventana deslizante de 14 días** que garantiza recuperación automática ante fallos puntuales de descarga:

Descarga estándar

```
1 # Últimas 2 semanas, hora solar 12:00 (por defecto)
2 python3 acces_api/scheduled_download.py
3
4 # Personalizar ventana y hora
5 python3 acces_api/scheduled_download.py --days 30 --hour 15
6
7 # Descarga masiva histórica (todas las horas)
8 python3 acces_api/scheduled_download.py --days 90 --hour all
```

El sistema evita duplicados comprobando la existencia del fichero antes de descargar y utiliza un nombre determinista: `UNIX_YYYYMMDD_HHMMSS_REF.jpg`.

2.3.1 Automatización con Cron

Para descarga diaria autónoma:

`/etc/cron.d/cvlit-download`

```
1 # Descarga diaria a las 13:00h (12h solar + margen de publicación API)
2 0 13 * * * cd /home/nickhernd/Desktop/cv-lit && \
3   python3 acces_api/scheduled_download.py >> data/logs/cron.log 2>&1
```

Capítulo 3

Arquitectura del Sistema

3.1 Stack tecnológico

Cuadro 3.1: Dependencias principales

Componente	Versión	Rol
Python	3.11	Runtime principal
FastAPI	≥ 0.110	API REST backend
OpenCV	4.x	Procesamiento de imagen
NumPy/SciPy	–	Álgebra matricial
SAM (Meta AI)	–	Segmentación semántica
PyTorch	≥ 2.0	Backend SAM
GDAL/pyproj	–	Transformaciones EPSG
Vue 3 + Vite	–	Interfaz web
Tailwind CSS	3.x	Estilos

3.2 Estructura de directorios

Árbol del repositorio

```
1 cv-lit/
2 +-- acces_api/           # Módulo Obscape API
3 |   +-- obscape_api.py   # Cliente REST
4 |   +-- scheduled_download.py
5 +-- backend/             # Servidor FastAPI
6 |   +-- main.py          # App principal + routers
7 |   +-- batch_alignment.py # Pipeline de alineación masiva [NUEVO]
8 |   +-- config.py        # Constantes globales
9 |   +-- routers/
10 |       +-- alignment.py # Endpoints unitarios de alineación
11 +-- calibration/         # Perfiles y máscaras por cámara
12 |   +-- cam_N_H.npy       # Matriz H 3x3 en float64
13 |   +-- cam_N_profile.json # Metadatos, RMSE, fecha
14 |   +-- roi_config.json   # ROIs por cámara
```

```

15 |   +-- alignment_masks.json# Zonas SIFT estables  [NUEVO]
16 |   +-- diagnostics/        # Imágenes de diagnóstico
17 +-- data/                  # Imágenes y resultados (excl. Git)
18 |   +-- CAM_N/
19 |   |   +-- images/        # Originales
20 |   |   +-- aligned/      # Salida del commit de batch
21 |   +-- logs/
22 +-- docs/                  # Documentación técnica
23 |   +-- memoria.tex        # Esta memoria  [NUEVO]
24 |   +-- reports/
25 +-- frontend/             # Interfaz Vue 3
26 |   +-- src/components/
27 |       +-- App.vue
28 |       +-- Calibration.vue
29 |       +-- BatchAlignment.vue  [NUEVO]
30 +-- homography/
31 |   +-- align_images.py    # CLI standalone de alineación
32 +-- proces_images/        # Algoritmos de procesamiento
33 |   +-- calibration_tool.py
34 |   +-- extract_coastline.py
35 |   +-- segmentation_sam.py
36 |   +-- recalibrate.py
37 +-- scripts/              # Herramientas de operación
38 +-- tests/

```

3.3 Pipeline de procesamiento

La figura 3.1 muestra el grafo completo de procesamiento desde la adquisición hasta la exportación del GeoJSON georreferenciado.

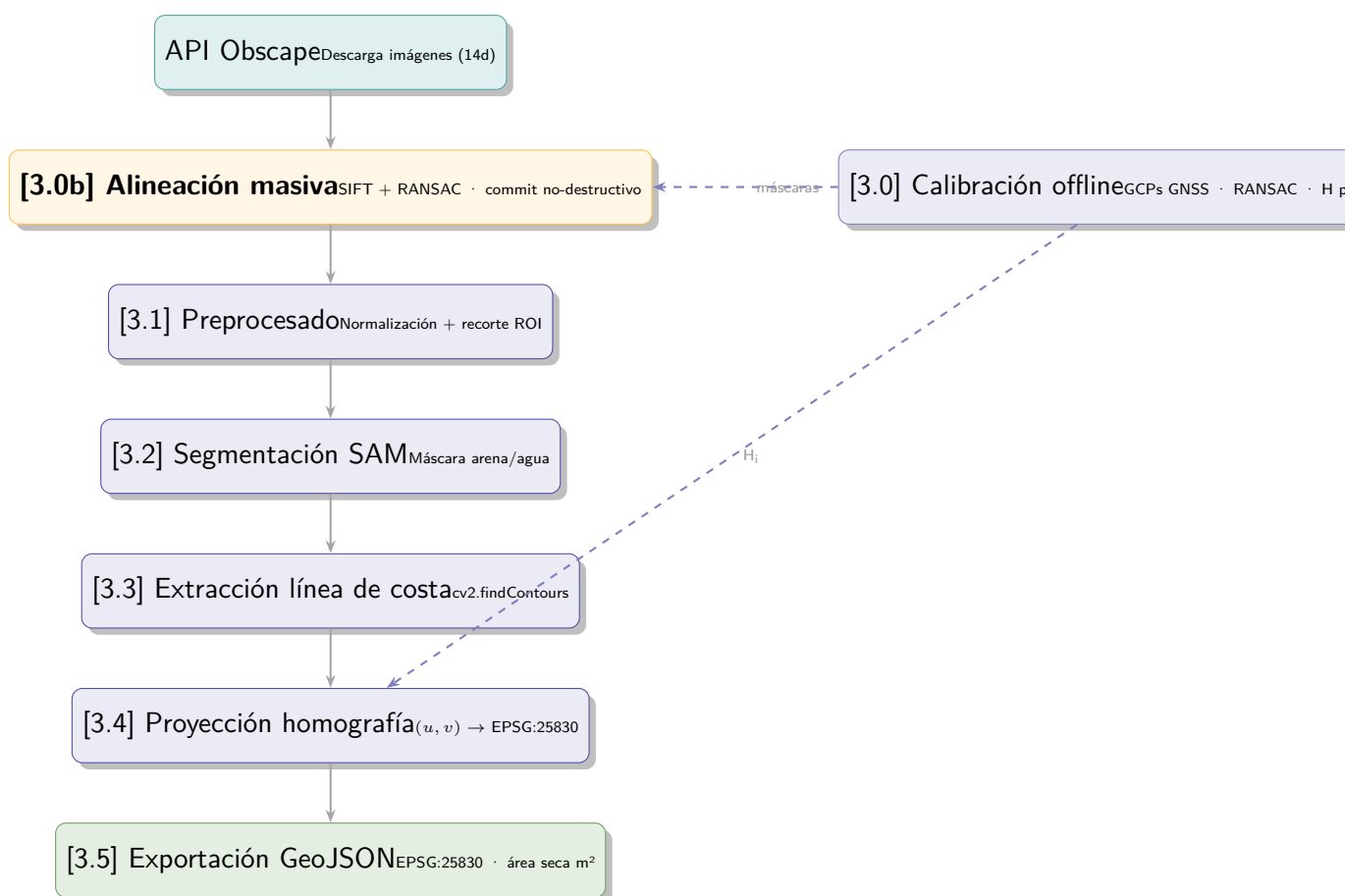


Figura 3.1: Pipeline completo de procesamiento cv-lit

Capítulo 4

Calibración Geométrica

4.1 Fundamento matemático

La calibración estima la **homografía planar** $\mathbf{H}$ (matriz 3×3) que transforma coordenadas de imagen (u, v) a coordenadas métricas (X, Y) en EPSG:25830:

$$\begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \sim \mathbf{H} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \quad (4.1)$$

La hipótesis de planaridad es válida dado que el objeto de interés (línea de costa) es aproximadamente plano a escala de cada cámara.

4.1.1 Corrección de distorsión radial

Antes de estimar $\mathbf{H}$, los puntos imagen se corrigen con los parámetros intrínsecos:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{D} = (k_1, k_2, p_1, p_2, k_3) \quad (4.2)$$

Implementado via `cv2.calibrateCamera()` con tablero de ajedrez o GCPs propios.

4.1.2 Estimación RANSAC

Estimación de homografía con RANSAC

```
1 H, mask = cv2.findHomography(  
2     pts_img,          # puntos imagen (u,v)  
3     pts_world,        # coordenadas UTM (X,Y) en EPSG:25830  
4     cv2.RANSAC,  
5     ransacReprojThreshold=5.0 # umbral de reproyección (px)  
6 )
```

4.2 Proceso de calibración por cámara

1. **Recepción de GCPs** del IEL: puntos de control con coordenadas (X, Y, Z) en EPSG:25830 y su correspondiente píxel imagen (u, v) . Mínimo 4 pares; recomendado ≥ 8 .
2. **Marcado interactivo** con `calibration_tool.py`:
 - Click izquierdo $\rightarrow$ punto de *calibración* (verde)
 - Click derecho $\rightarrow$ punto de *validación* (azul, no usado en H)
 - Tecla `h` $\rightarrow$ estimación RANSAC
 - Tecla `s` $\rightarrow$ guardar perfil
 - Tecla `p` $\rightarrow$ exportar imagen de diagnóstico
3. **Validación ciega**: el RMSE se calcula por separado sobre los puntos de validación (no vistos en RANSAC), garantizando una estimación honesta del error.
4. **Guardar perfil**: `calibration/cam_N_profile.json` con **H**, metadatos, RMSE calibración/validación y fecha.

4.3 Resultados de calibración (junio 2026)

Cuadro 4.1: Resultados de precisión por cámara

Cámara	GCPs	RMSE Cal. (m)	RMSE Val. (m)	RMSE Val. (px)	Estado
CAM 1	51	1.53	1.45	19.36	OK
CAM 2	39	2.64	3.45	21.09	Revisar
CAM 3	49	1.61	2.32	38.38	OK
CAM 4	49	2.78	3.06	100.01	Revisar
CAM 5	61	1.45	1.38	14.81	OK
CAM 6	69	6.14	4.74	130.62	Critico

Cámaras con error elevado

Los errores de CAM 4 (100 px) y CAM 6 (130 px) superan ampliamente el umbral de aceptación y sugieren una de estas causas:

- Distorsión radial no corregida (requiere parámetros intrínsecos reales).
- Inconsistencia en los puntos manuales (GCPs marcados con baja precisión).
- Movimiento de la cámara entre la toma de referencia y la calibración.

Acción recomendada: recalibrar con `recalibrate.py -cam 4` y `-cam 6` sobre imágenes actuales, revisando la distribución espacial de los GCPs.

4.4 Imágenes de diagnóstico

Las imágenes de la figura 4.1 muestran la reproyección de GCPs sobre las imágenes de referencia. Los vectores de error se representan magnificados $\times 10$.



(a) CAM 1 — RMSE val. 1.45 m



(b) CAM 2 — RMSE val. 3.45 m



(c) CAM 3 — RMSE val. 2.32 m



(d) CAM 4 — RMSE val. 3.06 m (revisar)



(e) CAM 5 — RMSE val. 1.38 m



(f) CAM 6 — RMSE val. 4.74 m (crítico)

Figura 4.1: Diagnóstico de calibración — reproyección de GCPs (vectores $\times 10$)

4.5 Recalibración rápida

Si la cámara sufre un ligero movimiento (viento, mantenimiento), se puede actualizar la homografía sin reintroducir coordenadas UTM:

Recalibración

```
1 # Arrastra los GCPs existentes sobre la imagen nueva
2 python3 proces_images/recalibrate.py --cam 1 --image /ruta/nueva_foto.jpg
```

Capítulo 5

Módulo de Alineación Masiva

5.1 Motivación

El flujo anterior validaba fotogramas de forma **unitaria**: una llamada API por imagen, sin visión global del lote. Esto presentaba tres problemas:

1. **Escalabilidad**: con 24 imágenes/día por 6 cámaras, revisar manualmente cada alineación era inviable.
2. **Detección de deriva**: sin comparativa global era difícil detectar si una cámara había girado ligeramente con el tiempo.
3. **Objetos dinámicos**: SIFT sin restricciones detectaba gaviotas y olas como features, produciendo homografías sesgadas.

5.2 Arquitectura de estados

El módulo implementa una **máquina de estados no-destructiva** de 5 fases:

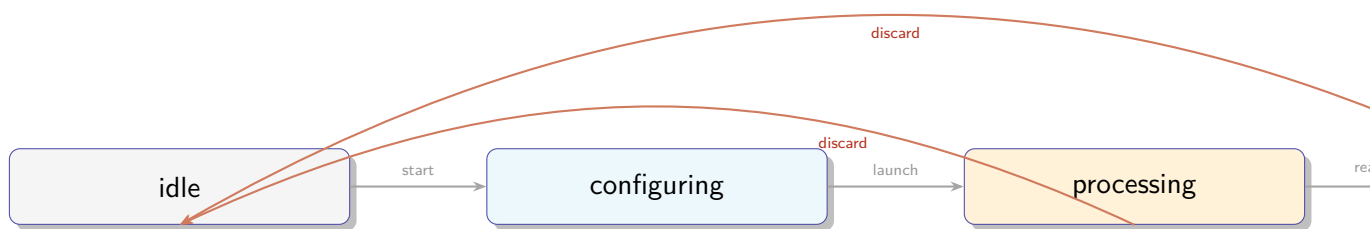


Figura 5.1: Máquina de estados del módulo BatchAlignment

Principio de no-destructividad

Todo el trabajo intermedio vive en `/tmp/cv_lit_batch/{job_id}/`. Los originales de cada cámara **nunca** se modifican. La escritura permanente ocurre **únicamente** al llamar al commit explícito.

5.2.1 Estructura del directorio temporal por job

```

1 /tmp/cv_lit_batch/{job_id}/
2   aligned/      ← imágenes con warpPerspective aplicado (staging)
3   diff/         ← mapas de delta por imagen
4   blend/        ← blend 50/50 en escala de grises

```

5.3 Pipeline de alineación: algoritmo

5.3.1 Parámetros configurables

Cuadro 5.1: Parámetros del pipeline de alineación (`batch_alignment.py`)

Constante	Valor	Descripción
SIFT_FEATURES	3 000	Keypoints máximos por imagen
LOWE_RATIO	0.75	Ratio test de Lowe para filtrar matches
MIN_INLIERS	15	Inliers mínimos RANSAC (con máscara)
MIN_INLIERS_FULL	30	Inliers mínimos sin máscara (fallback)
RANSAC_THRESH	5.0 px	Umbral de reproyección RANSAC

5.3.2 Estrategia de dos pasadas

Para combinar la robustez de las máscaras con la resiliencia ante zonas escasas de features:

Lógica de dos pasadas (pseudocódigo)

```

1  # Pasada 1: con máscara de zonas estables
2  H, inliers, n_good, fail = _try_align(gray, ref_kp, ref_des,
3                                     mask=feat_mask,
4                                     threshold=MIN_INLIERS)  # 15
5  if inliers >= MIN_INLIERS:
6      result.used_mask = True
7      return H, inliers
8
9  # Pasada 2 (fallback): toda la imagen, umbral más alto
10 H, inliers, n_good, fail = _try_align(gray, ref_kp_full, ref_des_full,
11                                     mask=None,
12                                     threshold=MIN_INLIERS_FULL)  # 30
13 result.used_mask = False

```

5.3.3 Flujo completo del pipeline

1. Carga imagen base → extrae keypoints con SIFT (con y sin máscara).
2. Por cada imagen del lote (tarea de fondo asíncrona):
 - a) Detecta features → matching FLANN con ratio test de Lowe (0.75).
 - b) RANSAC → homografía $\mathbf{H}$ (dos pasadas).
 - c) `cv2.warpPerspective` → imagen alineada → `aligned/`.
 - d) `_compute_diff_map` → mapa de delta → `diff/`.
 - e) `_blend_grayscale` → blend 50/50 → `blend/`.
 - f) `_compute_mean_shift` → métrica escalar δ_{px} .
 - g) Actualiza `job.progress_current`.
3. `job.status = ready` al finalizar.

5.4 Renderizado del mapa de delta

El mapa de delta es una imagen compuesta de tres capas superpuestas que proporciona tanto información visual intuitiva como métricas cuantitativas.

5.4.1 Capa 1 — Heatmap de diferencia absoluta

$$\Delta_{\text{abs}}(i, j) = \frac{1}{3} \sum_{c \in \{R, G, B\}} |I_{\text{ref}}(i, j, c) - I_{\text{ali}}(i, j, c)| \quad (5.1)$$

Se normaliza y se aplica `COLORMAP_HOT` de OpenCV:

Cuadro 5.2: Leyenda del heatmap HOT

Color	Significado
Negro	$\Delta = 0$ — alineación perfecta
Rojo/naranja	Error de intensidad moderado
Amarillo	Error alto
Blanco	Error severo — zona mal alineada

5.4.2 Capa 2 — Divergencia de bordes (overlay cian)

```

1 edges_ref = cv2.Canny(ref_gray, 40, 120)
2 edges_ali = cv2.Canny(aligned_gray, 40, 120)
3 edge_diff = cv2.dilate(cv2.absdiff(edges_ref, edges_ali), kernel_5x5)
4 composite[edge_diff > 0] = (255, 255, 0) # cian en BGR

```

Los bordes estructurales que **no coinciden** entre referencia y alineada aparecen en **cian**, identificando exactamente qué objeto se desplazó y en qué dirección.

5.4.3 Capa 3 — Métrica escalar δ_{px} (flujo óptico Farneback)

$$\delta_{\text{px}} = \text{mediana} \left(\sqrt{v_x(i, j)^2 + v_y(i, j)^2} \right) \quad (5.2)$$

donde (v_x, v_y) es el campo de flujo óptico residual post-warp calculado con Farneback. Mide cuántos píxeles de desplazamiento sistemático quedan tras la homografía.

Umbral de alerta en la UI: $\delta_{\text{px}} > 3$.

5.5 API REST del módulo

Cuadro 5.3: Endpoints del router `/api/batch/`

Método	Ruta	Descripción
POST	<code>/start</code>	Encola job + lanza pipeline en background
GET	<code>/\{id\}/status</code>	Polling de progreso (fase processing)
GET	<code>/\{id\}/results</code>	Lista completa de métricas (fase ready)
GET	<code>/\{id\}/preview/original/\{fn\}</code>	Imagen original sin transformar
GET	<code>/\{id\}/preview/aligned/\{fn\}</code>	Imagen alineada en staging
GET	<code>/\{id\}/preview/diff/\{fn\}</code>	Mapa de delta compuesto
GET	<code>/\{id\}/preview/blend/\{fn\}</code>	Blend 50/50 en escala de grises
PATCH	<code>/\{id\}/approve/\{fn\}</code>	Override de aprobación individual
POST	<code>/\{id\}/commit</code>	Two-phase commit → <code>CAM_X/aligned/</code>
DELETE	<code>/\{id\}</code>	Descarta job y elimina temp dir

5.6 Modelos de dominio

Dataclasses principales (`batch_alignment.py`)

```

1 @dataclass
2 class AlignmentResult:
3     filename: str
4     status: str # "ok" / "failed" / "reference"
5     inliers: int # inliers RANSAC
6     mean_shift_px: float # mediana flujo óptico residual (px)
7     H: Optional[List] # homografía 3x3 como lista plana

```

```

8     approved:      bool = True      # override de aprobación por imagen
9     fail_reason:   Optional[str] = None # no_features/low_matches/ransac_failed
10    used_mask:      bool = True      # True = pasada con máscara activa
11
12    @dataclass
13    class BatchJob:
14        job_id:      str
15        cam_id:      int
16        base_filename: str
17        image_filenames: List[str]
18        status:      str #
19        ↪ pending/processing/ready/committed/failed/discarded
20        results:     Dict[str, AlignmentResult]
21        progress_current: int
22        progress_total: int

```

5.7 Flujo de commit (two-phase write)

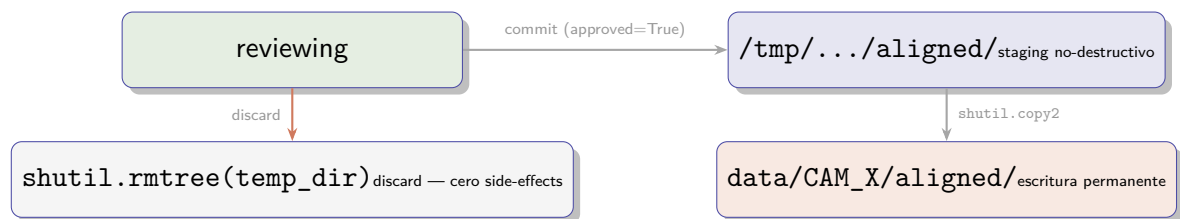


Figura 5.2: Flujo de commit no-destrutivo

Capítulo 6

Máscaras de Zonas Estables SIFT

6.1 Problema: objetos dinámicos en SIFT

Sin restricciones de zona, SIFT detecta features en **cualquier región** de la imagen, incluyendo objetos dinámicos que producen correspondencias erróneas:

- **Gaviotas y pájaros** en vuelo (zona de cielo/horizonte bajo)
- **Olas y espuma** de mar (bordes dinámicos muy contrastados)
- **Personas y bañistas** en la playa
- **Sombras dinámicas** de nubes

Resultado: matches entre objetos en posiciones distintas entre tomas → homografía sesgada → imágenes “giradas” o desplazadas incorrectamente.

6.2 Solución: máscaras de regiones estables

El fichero `calibration/alignment_masks.json` define zonas con **coordenadas fraccionarias** (0,0...1,0) independientes de la resolución real de la imagen:

$$x_{\text{frac}} = \frac{x_{\text{px}}}{W_{\text{imagen}}}, \quad y_{\text{frac}} = \frac{y_{\text{px}}}{H_{\text{imagen}}} \quad (6.1)$$

`alignment_masks.json` - estructura

```
1 {
2   "CAM_1": {
3     "regions": [
4       {"label": "horizonte_mar", "x0": 0.00, "y0": 0.31, "x1": 1.00, "y1":
        ↪ 0.40},
5       {"label": "arena_baja", "x0": 0.00, "y0": 0.73, "x1": 0.60, "y1":
        ↪ 0.88},
6       {"label": "vegetacion_dcha", "x0": 0.60, "y0": 0.73, "x1": 1.00, "y1":
        ↪ 0.92}
7     ]
8   }
```

9

}

6.3 Zonas por cámara

Cuadro 6.1: Configuración de máscaras SIFT por cámara

Cámara	Zona 1	Zona 2	Justificación
CAM 1	Horizonte mar (y=0.31–0.40)	Arena baja + Vegetación dcha	Excluye cielo con gaviotas
CAM 2	Horizonte (y=0.26–0.40)	El Peñón (x=0.58–0.88)	El Peñón de Ifach es referencia fija excelente
CAM 3	Horizonte (y=0.34–0.48)	Edificio dcha (x=0.72–1.00)	Edificio como anclaje vertical
CAM 4	Horizonte (y=0.30–0.44)	Edificio izq (x=0.00–0.28)	Edificio como anclaje vertical
CAM 5	Horizonte (y=0.28–0.42)	Zona izq (x=0.00–0.32)	
CAM 6	Horizonte (y=0.32–0.46)	Edificios izq (x=0.00–0.35)	

6.4 Corrección CAM_1: problema de gaviotas



(a) Máscara anterior — zona “horizonte” comienza en $y = 0,24$, alcanzando el cielo con gaviotas
(b) Máscara corregida — **horizonte_mar** comienza en $y = 0,31$ (mar), más arena baja y vegetación

Figura 6.1: Comparativa de máscaras SIFT para CAM 1 (imagen 4608×2682 px)

Por qué CAM_1 es problemática

La cámara apunta hacia el norte sobre la playa. En imágenes de mañana, 5–10 gaviotas sobrevuelan habitualmente en la banda $y \approx 0,14 \dots 0,30$ (cielo bajo). SIFT detecta las aves como features de alto contraste con gran confianza. Al estar en posiciones distintas entre tomas, generan matches cruzados que sesgan la

homografía.

La corrección: `horizonte_mar` ($y = 0,31 \dots 0,40$) está en la franja de agua cerca del horizonte, donde los features son reproducibles y estables.

6.4.1 Coordenadas en píxeles (imagen 4608×2682)

Cuadro 6.2: Conversión fracción $\rightarrow$ píxeles para CAM_1

Zona	Esquina sup-izq (px)	Esquina inf-der (px)
<code>horizonte_mar</code>	(0, 831)	(4608, 1072)
<code>arena_baja</code>	(0, 1957)	(2764, 2360)
<code>vegetacion_dcha</code>	(2764, 1957)	(4608, 2467)

6.5 Cómo ajustar una máscara

1. Abrir una imagen real de la cámara en un visor que muestre coordenadas de píxel.
2. Identificar las regiones estables (sin gaviotas, olas ni personas).
3. Anotar las coordenadas en píxeles de las esquinas.
4. Convertir a fracciones: $x_f = x_{\text{px}}/W$, $y_f = y_{\text{px}}/H$.
5. Editar `calibration/alignment_masks.json`.
6. Ejecutar el script de verificación:

Script de verificación visual de máscaras

```

1  import cv2, json, numpy as np
2
3  IMG   = "proces_images/data/camera1/IMAGEN_REF.jpg"
4  MASKS = "calibration/alignment_masks.json"
5  CAM   = "CAM_1"
6
7  img = cv2.imread(IMG)
8  h, w = img.shape[:2]
9  overlay = img.copy()
10
11  with open(MASKS) as f:
12      cfg = json.load(f)
13
14  for r in cfg[CAM]["regions"]:
15      x0, y0 = int(r["x0"]*w), int(r["y0"]*h)
16      x1, y1 = int(r["x1"]*w), int(r["y1"]*h)
17      cv2.rectangle(overlay, (x0,y0), (x1,y1), (0,255,0), -1)
18      cv2.putText(overlay, r["label"], (x0+10, y0+30),
19                  cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,0,0), 2)

```

```
20
21 result = cv2.addWeighted(img, 0.5, overlay, 0.5, 0)
22 cv2.imwrite("/tmp/mask_check.jpg", result)
23 print(f"Guardado en /tmp/mask_check.jpg  ({w}x{h})")
```

Capítulo 7

Interfaz Web

7.1 Arquitectura frontend

La interfaz web está construida con **Vue 3** + **Vite** + **Tailwind CSS**, sin gestión de estado global (no Pinia): cada componente mantiene su propio estado reactivo con `ref()` y `reactive()`.

Para iniciar el entorno de desarrollo:

```
1 ./start_dev.sh
2 # Backend: http://localhost:8000
3 # Frontend: http://localhost:5173
```

7.2 Componente BatchAlignment.vue

Componente principal del módulo de alineación masiva. Integrado en el **paso 3** del flujo de calibración (`Calibration.vue`).

7.2.1 Variables de estado

```
1 const phase = ref('idle') //
  ↳ idle/configuring/processing/reviewing/committed
2 const jobId = ref(null) // UUID del job backend activo
3 const results = ref([]) // Array de AlignmentResult del lote
4 const selected = ref(null) // filename seleccionado en filmstrip
5 const viewMode = ref('diff') // 'diff' | 'blend' | 'aligned'
```

7.2.2 Props y eventos

Cuadro 7.1: Props del componente BatchAlignment

Prop	Tipo	Descripción
camId	Number	ID de cámara (1–6)
imageList	Array	Lista de imágenes disponibles
initialBase	String	Imagen base por defecto (del perfil de calibración)
Emit	Payload	Descripción
notify	(msg, type)	Toast del sistema
committed	{jobId, committed}	Lote confirmado con éxito
discard	—	Usuario descartó el job

7.2.3 Layout de la fase reviewing

Vista de revisión - esquema

```

1  +-----+-----+-----+-----+
2  | RESUMEN          | [Mapa delta] [Blend 50/50] [Alineada] |
3  | Aprobadas: 44    | +-----+-----+-----+ |
4  | Omitidas:  3     | |                               | |
5  | Fallidas:  0     | | ORIGINAL          | DELTA / BLEND / ALI | |
6  +-----+-----+ | <img>          | <img>          | |
7  | FILMSTRIP        | |                               | |
8  | img_001 OK       | +-----+-----+-----+ |
9  | img_002 ~2.1px   | | Leyenda: negro=OK rojo=error cian=bordes | |
10 | img_003 FAIL     | |                               | |
11 | img_004 SKIP     | | inliers: 152 delta: 1.4 px [ Aprobada] | |
12 | ...              | |                               | |
13 +-----+-----+-----+-----+
14 | [Confirmar lote (44)]                [Descartar sin guardar] |
15 +-----+-----+-----+-----+

```

7.2.4 Indicadores del filmstrip

Cuadro 7.2: Código de color en el filmstrip

Color	Estado	Condición
Azul	Referencia	Imagen base seleccionada
Verde	OK	$\delta_{px} \leq 3$
Amarillo	Drift	$\delta_{px} > 3$
Ámbar	Desaprobada	<code>approved = false</code> (manual)
Rojo	Fallida	RANSAC insuficiente

7.2.5 Polling de progreso

El frontend consulta el estado del job cada **800 ms** durante la fase de procesamiento:

```

1 function _poll() {
2   // GET /api/batch/{jobId}/status
3   if (data.status === 'ready') {
4     clearInterval(pollTimer)
5     await _loadResults() // carga métricas completas
6     phase.value = 'reviewing'
7   }
8   if (data.status === 'failed') {
9     clearInterval(pollTimer)
10    emit('notify', data.error, 'error')
11    phase.value = 'configuring'
12  }
13  progress.value = data.progress_current / data.progress_total
14 }
```

7.3 Flujo completo de uso

1. Dashboard → seleccionar cámara → pestaña *Calibración* → paso 3.
2. Elegir imagen base de referencia del lote.
3. Pulsar “Alinear lote completo” → barra de progreso en tiempo real.
4. Revisar filmstrip: verde = OK, rojo = error.
5. Para cada imagen roja o amarilla: inspeccionar mapa de delta / blend.
6. Desaprobar imágenes concretas (opcional) con toggle en el visor.
7. Pulsar “Confirmar lote” → imágenes aprobadas a `data/CAM_X/aligned/`.
8. Las imágenes originales en `data/CAM_X/images/` quedan **intactas**.

Capítulo 8

Región de Interés (ROI)

8.1 Criterios de delimitación

Para optimizar la segmentación con SAM y reducir ruido en la extracción de línea de costa, cada cámara tiene una ROI que excluye:

- **Cielo:** píxeles por encima del horizonte.
- **Infraestructura:** paseos marítimos, espigones, edificios fuera de la zona de interés.
- **Mar profundo:** zona sin rotura de onda donde la línea de costa no es visible.

8.2 Coordenadas por cámara

Las ROIs se almacenan en `calibration/roi_config.json`:

Cuadro 8.1: ROIs preliminares por cámara (a ajustar con imágenes de producción)

Cámara	ROI (x_0, y_0, x_1, y_1) px	Justificación
CAM 1	[0, 400, 1920, 1080]	Recorte superior (horizonte $\approx y = 400$)
CAM 2	[0, 350, 1920, 1080]	Horizonte más bajo
CAM 3	[0, 450, 1920, 1080]	Ángulo con más cielo visible
CAM 4	[0, 400, 1920, 1080]	
CAM 5	[0, 380, 1920, 1080]	
CAM 6	[0, 420, 1920, 1080]	

Nota de actualización

Estos valores son preliminares y se ajustarán tras la primera recepción de imágenes de producción de todas las cámaras de Guardamar. Verificar superposición ROI sobre imagen real para confirmar que no se pierde zona de intermarea.

Capítulo 9

Segmentación y Extracción de Línea de Costa

9.1 Segmentación semántica con SAM

Se utiliza **Segment Anything Model (SAM)** de Meta AI en modo *zero-shot*: sin datos de entrenamiento específicos de Guardamar.

- **Modo de uso:** prompt basado en puntos semilla o bounding boxes de la ROI.
- **Salida:** mapa probabilístico arena seca / arena húmeda / agua.
- **Post-SAM:** umbral adaptativo + filtrado morfológico → máscara binaria.

Segmentación (extracto segmentation_sam.py)

```
1 from segment_anything import sam_model_registry, SamAutomaticMaskGenerator
2
3 sam = sam_model_registry["vit_h"](checkpoint=SAM_CHECKPOINT)
4 mask_generator = SamAutomaticMaskGenerator(
5     sam,
6     points_per_side=32,
7     pred_iou_thresh=0.86,
8     stability_score_thresh=0.92
9 )
10 masks = mask_generator.generate(roi_img)
```

9.2 Extracción de línea de costa

1. Cálculo de contornos sobre la máscara binaria: `cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`
2. Selección del contorno correspondiente al borde arena húmeda – arena seca.
3. Resultado: polilínea en coordenadas píxel (u, v) .

9.3 Proyección a EPSG:25830

Aplicación de la homografía (4.1) para transformar cada punto (u, v) de la polilínea a coordenadas métricas:

```

1 def pixel_to_utm(H, pts_px):
2     """pts_px: Nx2 array de (u,v). Retorna Nx2 en UTM EPSG:25830."""
3     pts_h = np.hstack([pts_px, np.ones((len(pts_px), 1))])
4     result = (H @ pts_h.T).T
5     return result[:, :2] / result[:, 2:3]    # normalización homogénea

```

9.4 Exportación GeoJSON

Estructura del GeoJSON de salida

```

1 {
2     "type": "FeatureCollection",
3     "crs": {
4         "type": "name",
5         "properties": { "name": "EPSG:25830" }
6     },
7     "features": [{
8         "type": "Feature",
9         "geometry": {
10             "type": "LineString",
11             "coordinates": [[711234.5, 4209876.3], ["..."]]
12         },
13         "properties": {
14             "ID_Camara": 1,
15             "Timestamp": "2026-04-30T12:00:00Z",
16             "Confianza_IA": 0.93,
17             "Area_Seca_m2": 14250.7
18         }
19     }]
20 }

```

Capítulo 10

Validación y Métricas de Calidad

10.1 Estrategia de validación

Comparación de la línea detectada contra **transectos GNSS independientes** del IEL: puntos **no usados** en la calibración de homografía.

10.2 Métricas

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N d_i^2} \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N d_i \quad (10.1)$$

donde d_i es la distancia mínima en metros del punto GNSS i a la polilínea detectada.

10.3 Criterios de aceptación

Cuadro 10.1: Umbrales de calidad del sistema

Métrica	Objetivo	Condición
RMSE geométrico	< 1,5 m	Condiciones estándar (12:00h, sin interferencias)
MAE geométrico	< 1,0 m	Objetivo secundario
Cobertura temporal	≥ 90 %	Imágenes a 12:00h procesadas sin error
δ_{px} alineación	< 3 px	Drift residual post-homografía

10.4 Factores de error a monitorizar

- **Calidad de imagen:** niebla, reflejo solar, lluvia, espuma excesiva.

- **Nivel de marea:** afecta la posición real de la línea (calibrar en diferentes estados de marea).
- **Deriva de cámara:** movimiento del soporte por viento, mantenimiento u otras causas.
- **Precisión GCPs:** exactitud del levantamiento GNSS del IEL (instrumental + propagación de error).

10.5 Validación en QGIS

1. Configurar CRS del proyecto en **EPSG:25830**.
2. Cargar GCPs del IEL como capa de puntos.
3. Arrastrar los GeoJSON de `data/processed/lines/` al proyecto.
4. Usar “Distancia a la polilínea más cercana” para calcular d_i .
5. Comparar con ortofoto PNOA (WMS/WMTS del IGN) para validación visual.

Capítulo 11

Estado del Proyecto y Roadmap

11.1 Estado actual (junio 2026)

Cuadro 11.1: Estado de hitos del proyecto

Mes	Objetivo	Estado	Nota
1	Preparación datos + diseño interfaces	Hecho	Completado
2	Calibración geométrica	Hecho	6 perfiles generados
3	Segmentación (prototipo)	Hecho	SAM funcional
4	Extracción línea + georreferenciación	Hecho	GeoJSON en EPSG:25830
5	Validación y robustez	Parcial	CAM 4 y 6 pendientes
6	Integración y entrega	Pendiente	Próximo hito

11.2 Completado en la iteración de junio 2026

- Módulo de alineación masiva no-destructiva (`batch_alignment.py`)
 - Router FastAPI `/api/batch/` con 10 endpoints
 - Pipeline async SIFT+FLANN+RANSAC sobre lote completo
 - Mapa de delta compuesto (heatmap HOT + edge-diff cian + métrica Farneback)
 - Two-phase commit: staging `/tmp/` → escritura permanente sólo en commit
- Interfaz de validación por lotes (`BatchAlignment.vue`)
- Corrección de máscaras SIFT para CAM 1 (problema de gaviotas)
- Integración en paso 3 del flujo de calibración (`Calibration.vue`)
- Documentación completa (esta memoria)

11.3 Fase 3 — Automatización y robustez (mes 5)

1. **Procesamiento por lotes:** integrar backend para procesar todas las imágenes de 12:00h.
2. **Filtros de calidad:** rechazo automático de imágenes con niebla o baja confianza de SAM.
3. **Persistencia GeoJSON:** guardar histórico de líneas de costa en base de datos/sistema de ficheros.
4. **Recalibración CAM 4 y CAM 6:** reducir RMSE por debajo de 1.5 m.

11.4 Fase 4 — Entrega e integración (mes 6)

1. Dashboard de histórico: gráficas de evolución del área seca.
2. Exportación masiva: informes mensuales de variación de línea de costa.
3. Manual de usuario: documentación final para el personal del Ayuntamiento.

Capítulo 12

Guía de Operación

12.1 Inicio rápido

Arrancar el sistema completo

```
1 # 1. Activar entorno Conda
2 conda activate cv-lit
3
4 # 2. Iniciar backend + frontend en paralelo
5 ./start_dev.sh
6 # Backend: http://localhost:8000 (FastAPI + docs en /docs)
7 # Frontend: http://localhost:5173 (Vue 3 + Vite)
```

12.2 Descarga de imágenes

```
1 # Descarga estándar (últimas 2 semanas, 12:00h)
2 python3 acces_api/scheduled_download.py
3
4 # Diagnóstico de estado de cámaras
5 python3 scripts/verify_setup.py
```

12.3 Calibración

```
1 # Herramienta completa de calibración
2 python3 proces_images/calibration_tool.py --cam 1 --image /ruta/foto.jpg
3
4 # Lanzador visual rápido (CAM 1, imagen más reciente)
5 python3 scripts/visualizar_calibracion.py
6
7 # Recalibración rápida (cámara movida levemente)
```

```
8 python3 proces_images/recalibrate.py --cam 1 --image /ruta/nueva_foto.jpg
9
10 # Verificar motor matemático
11 python3 proces_images/test_calibration_logic.py
```

12.4 Alineación masiva (CLI)

Para usar el módulo de alineación sin interfaz web:

```
1 # Alinear todas las imágenes de una carpeta respecto a la primera
2 python3 homography/align_images.py \
3     --input /ruta/imagenes/ \
4     --output /ruta/salida/ \
5     --cam 1 # aplica máscaras SIFT de CAM_1
6
7 # Especificar imagen de referencia manualmente
8 python3 homography/align_images.py \
9     --input /ruta/imagenes/ \
10    --output /ruta/salida/ \
11    --ref imagen_base.jpg \
12    --cam 2
```

Salidas:

- /output/aligned/ — imágenes alineadas
- /output/diagnostics/ — visualizaciones de matches
- /output/homographies.csv — métricas y matrices H por imagen

12.5 Pipeline de extracción

```
1 # Prototipo completo: ROI → Segmentación SAM → Línea de costa
2 python3 proces_images/test_mes3_pipeline.py --cam 1
```

Capítulo 13

Instalación y Configuración

13.1 Requisitos del sistema

Cuadro 13.1: Requisitos hardware y software

Componente	Requisito mínimo
SO	Linux (Ubuntu 22.04+) o Windows 10+
Python	3.11+
RAM	8 GB (16 GB recomendado para SAM)
GPU	CUDA opcional pero recomendada para SAM
Almacenamiento	50 GB para datos (6 cámaras × 24h/día)
Node.js	18+ (para frontend)

13.2 Instalación del entorno Python

```
1 conda create -n cv-lit python=3.11
2 conda activate cv-lit
3
4 # Visión + álgebra
5 pip install opencv-python numpy scipy pandas
6
7 # Backend API
8 pip install fastapi uvicorn python-multipart
9
10 # Segmentación SAM
11 pip install torch torchvision
12 pip install git+https://github.com/facebookresearch/segment-anything.git
13
14 # Geo
15 pip install gdal pyproj shapely
```

13.3 Instalación del frontend

```

1 cd frontend
2 npm install
3 npm run dev      # modo desarrollo
4 npm run build    # build de producción (dist/)

```

13.4 Configuración de la API Obscape

Las credenciales de acceso a la API se configuran en `acces_api/obscape_api.py`:

Cuadro 13.2: Parámetros de conexión Obscape

Parámetro	Valor
Base URL	<code>https://obscape.com/portal/api/v3/api</code>
Username	<code>fuster</code>
Portal	<code>https://obscape.com/portal</code>

Apéndice A

Convenciones de código y Git

A.1 Estrategia de ramas

- **main**: código estable y validado.
- **feature/nombre-modulo**: nuevas funcionalidades.
- **fix/descripcion-error**: correcciones.
- **docs/nombre-doc**: documentación.

Todo cambio debe pasar por Pull Request antes de integrar en **main**.

A.2 Nomenclatura de ficheros

Las imágenes descargadas usan el formato determinista:

`UNIX_YYYYMMDD_HHMMSS_REFERENCIA.jpg`

A.3 Proyección obligatoria

Toda coordenada geográfica en el sistema debe estar en **EPSG:25830**. Nunca almacenar en WGS84 (EPSG:4326) salvo para visualización en APIs de mapas.

Apéndice B

Glosario

EPSG:25830	ETRS89 / UTM zona 30N. Sistema de referencia cartográfico oficial para la Península Ibérica.
GCP	<i>Ground Control Point</i> . Punto de control terrestre con coordenadas conocidas.
Homografía	Transformación proyectiva planar entre dos planos (imagen y terreno).
IEL	Instituto de Ecología Litoral. Suministra los GCPs GNSS del proyecto.
RANSAC	<i>Random Sample Consensus</i> . Estimador robusto de parámetros geométricos.
ROI	<i>Region of Interest</i> . Zona de la imagen relevante para el análisis.
SAM	<i>Segment Anything Model</i> . Modelo de segmentación de Meta AI.
SIFT	<i>Scale-Invariant Feature Transform</i> . Detector de puntos de interés invariante a escala y rotación.
RMSE	<i>Root Mean Square Error</i> . Error cuadrático medio.
MAE	<i>Mean Absolute Error</i> . Error medio absoluto.
Farneback	Algoritmo de flujo óptico denso para estimar campo de desplazamiento entre dos imágenes.
Inliers	Correspondencias que se ajustan al modelo estimado por RANSAC.
Two-phase commit	Patrón de escritura en dos fases: staging temporal → confirmación permanente.
Drift	Desplazamiento residual de píxeles tras la corrección de homografía.

Apéndice C

Historial de cambios

Cuadro C.1: Historial de versiones del sistema

Fecha	Componente	Cambio
Ene 2026	API client	Cliente Obscape + descarga con deduplicación
Feb 2026	Calibración	<code>calibration_tool.py</code> : intrínsecos + validación ciega
Mar 2026	Calibración	<code>recalibrate.py</code> : recalibración por arrastre de GCPs
Abr 2026	Segmentación	Prototipo SAM + extracción de línea de costa
May 2026	Georreferenciación	Proyección a EPSG:25830 + exportación GeoJSON
May 2026	Calibración	Ejecución real con GCPs del IEL (6 cámaras)
Jun 2026	Backend	<code>batch_alignment.py</code> : pipeline asíncrono + 10 endpoints
Jun 2026	Frontend	<code>BatchAlignment.vue</code> : máquina de estados 5 fases
Jun 2026	Máscaras SIFT	<code>alignment_masks.json</code> : zonas estables por cámara
Jun 2026	CAM 1	Corrección máscara: excluir cielo con gaviotas ($y=0.24 \rightarrow 0.31$)
Jun 2026	Documentación	Esta memoria técnica en L ^A T _E X